```
»       when "Credit card"
»         payment_method = :credit_card
»         month, year = pay_type_params[:expiration_date].split(//)
»         payment_details[:cc_num] = pay_type_params[:credit_card_number]
»         payment_details[:expiration_month] = month
»         payment_details[:expiration_year] = year
»       when "Purchase order"
»         payment_method = :po
»         payment_details[:po_num] = pay_type_params[:po_number]
»       end
»
»       payment_result = Pago.make_payment(
»         order_id: id,
»         payment_method: payment_method,
»         payment_details: payment_details
»       )
»
»       if payment_result.succeeded?
»         OrderMailer.received(self).deliver_later
»       else
»         raise payment_result.error
»       end
»     end
    end
```

If you weren't concerned with how slow Pago's API is, you'd change the code in the `create` method of `OrdersController` to call `charge!`:

```
    if @order.save
      Cart.destroy(session[:cart_id])
      session[:cart_id] = nil
»     @order.charge!(pay_type_params) # do not do this
      format.html { redirect_to store_index_url, notice:
        'Thank you for your order.' }
```

Since you already know the call to Pago will be slow, you want it to happen in a background job so that users can see the confirmation message in their browser immediately without having to wait for the charge to actually happen. To do this, you must create an Active Job class, implement that class to call `charge!`, and then add code to the controller to execute this job. The flow looks like the figure shown.